

Deep Learning: Practical Work 1 - CNNs on MAMe Dataset

Clàudia Boixader & Óscar Cubeles

November 23rd, 2025

1 Description

This practical work is about evaluating the performance of several Deep CNN architectures for image classification using the MAMe dataset, which contains 37,407 artworks grouped into 29 material-based classes. We work with the 256×256 version of the images and train different models to analyze underfitting, overfitting, and the impact of Neural Networks architectural choices. The project tests two groups of architectures both standard CNNs and a non-standard CNNs architectures.

2 Architectures

2.1 Standard Architecture

Standard CNN architectures generally follow a classical design composed of repeated convolutional layers with ReLU activations and pooling layers to progressively reduce spatial dimensions. Their training typically relies on stochastic gradient descent complemented by practical enhancements such as image rescaling, data augmentation, dropout, and heuristic techniques like weight initialization rules or learning rate decay schedules, often motivated by empirical success rather than strict theoretical justification. Evaluation may also employ heuristics such as averaging predictions over multiple image crops, which can improve accuracy but sometimes reduce reproducibility.

This section presents the standard architectures used in this project, including the custom standard architecture model and the additional reference architecture. In total, two standard convolutional neural networks were developed, one specifically designed and implemented for this work, and another well-established architecture from the literature, AlexNet, which was reimplemented from scratch and evaluated to provide a baseline for comparison. The motivation for incorporating this additional model was twofold. First, it serves as reference baseline against which the performance of the custom standard model can be compared, allowing us to contextualize its results with respect to a present standard architecture in the literature. Second, evaluating multiple architectures of different complexities offers deeper insight into how model depth and structural design can affect the generalization ability and sensitivity to underfitting and overfitting.

2.1.1 AlexNet

For this project, AlexNet was reimplemented from scratch following the original architectural principles while being adapted to the characteristics of the dataset used. That is, RGB images of size 256×256 and to produce predictions over 29 target classes. The implementation follows the classical structure of AlexNet: an initial sequence of convolutional layers with large receptive fields and progressively increasing channel depth, interleaved with ReLU activations and max-pooling layers to progressively reduce spatial resolution. Batch Normalization was incorporated after each convolutional layer to stabilize training and improve convergence.

The network consists of five convolutional blocks, the first two including max-pooling operations, followed by three additional convolutional layers without pooling. The final part of the model includes a flattening operation and three fully connected layers, of which the first two contain 4096 units each and use ReLU activation with dropout to reduce overfitting. The last fully connected layer outputs the logits for the 29 classes. Compared to simpler architectures, AlexNet provides a deeper and higher-capacity model, acting as a meaningful historical baseline and enabling comparison with more modern architectures in terms of training dynamics and generalization performance.

2.1.2 Standard Architecture

The standard architecture designed for this project follows the classical principles of convolutional neural network design while incorporating more modern architectural elements to improve representational capacity and stability. The model begins with a 7×7 convolutional layer with stride 2 followed by batch normalization, ReLU activation, and max pooling, which rapidly reduces the input spatial dimensions while extracting low-level features.

A key component of this architecture is the use of residual blocks, inspired by the design philosophy introduced in ResNet architectures which are not strictly standard architectures. Each residual block consists of two convolutional layers followed by batch normalization and ReLU activations, together with a skip connection that bypasses the main transformation path. This shortcut mechanism helps mitigate vanishing gradients, enabling deeper feature extraction without compromising training stability.

The model is organized into three stages of residual blocks, progressively increasing the number of channels from 64 to 512 while reducing spatial resolution through strided convolutions. After the final stage, a global average pooling layer aggregates the spatial information into a fixed-size representation. The classifier consists of two fully connected layers with batch normalization, ReLU activation, and dropout to reduce overfitting, producing the final predictions for the 29 target classes. Despite incorporating residual blocks, this standard architecture lacks other variants characteristic of non-standard, such as attention mechanisms, thus, we consider this a standard architecture.

2.2 Non-Standard Architectures

As mentioned in the previous section, Standard CNNs typically adhere to a classical design of sequential convolutional layers with ReLU activations and pooling. The development of non-standard architectures is motivated by the need to move beyond this rigid sequential template and achieve superior performance and efficiency through intelligent structural modifications.

Non-standard models introduce sophisticated design elements to better exploit the information within the image tensor. Among others, these techniques include: Spatial Exploitation (factorized convolutions), Depth Exploitation (skip connections), Multi-Path Exploitation (Inception modules), Width Exploitation (increasing the number of feature channels), Feature Map Exploitation (Context Aggregation), Channel Boosting (Squeeze-and-Excitation block) and Attention-Based Structures (Vision Transformers).

To evaluate the impact of these advanced structural designs, our project tested three distinct non-standard architectures. The selected non-standard models were: ResNet18, in order to test the power of residual learning for depth exploitation; an InceptionV3 model, implemented via the Keras application library (`tf.keras.applications`) to showcase multi-path strategies; and a Vision Transformer, which was implemented from scratch to represent the modern paradigm of attention-based image processing.

2.2.1 ResNet18

The first non-standard architecture implemented was ResNet18. It was built completely from scratch to serve as a modern, deeper reference model that notably features residual learning. The model closely follows the original design proposed in the literature, while being adapted to the specific characteristics of the dataset used in this project. The network begins with an initial convolutional layer with a 7×7 kernel and stride 2, followed by batch normalization, ReLU activation, and max pooling, reducing the spatial resolution early in the pipeline.

The core of the model consists of four sequential stages, each composed of two residual *BasicBlock* modules. These blocks implement the standard residual structure: two consecutive 3×3 convolutional layers with batch normalization and ReLU activation, combined with a shortcut connection that allows the input to bypass the main transformation. When the spatial dimensions change due to strided convolutions, the shortcut path also includes a 1×1 convolution to ensure that the tensor shapes match. This residual formulation mitigates the vanishing gradient problem and enables deeper networks to train more effectively.

Across the four stages, the number of filters increases progressively from 64 to 512, with downsampling performed at the start of each new stage. After the final residual block, a global average pooling layer aggregates spatial information, and a fully connected output layer produces class logits for the 29 categories. Compared to AlexNet, ResNet18 offers a significantly deeper architecture with more sophisticated feature extraction capabilities, making it an effective benchmark for evaluating the relative performance and generalization of the custom standard model.

2.2.2 InceptionV3

The second non-standard model used was InceptionV3. Its fundamental innovation is the Inception Module, which performs multiple types of convolutions (1×1 , 3×3 , 5×5) and pooling in parallel, concatenating their outputs to capture features at various scales. To manage computational cost, InceptionV3 uses 1×1 convolutions for dimensionality reduction and employs factorized convolutions to make the overall model deeper yet more parameter-efficient.

For this model we decide to use transfer learning by using the already implemented code from keras.applications. The model was loaded with include_top=False, which removes the network’s original classification head, and weights=None, meaning only the architecture is loaded, and the weights are randomly initialized for training from scratch on the new dataset. The resulting base model acts as a powerful, deep feature extractor, converting the input image into highly abstract feature maps.

A custom classification head was subsequently appended to the base model’s output, adapting the extracted features to the specific task involving 29 target classes. The head commences with a GlobalAveragePooling2D layer to aggregate the spatial dimensions of the feature maps into a single vector. This is followed by a Dense layer (1024 units with ReLU activation) incorporating L2 Regularization to penalize large weights and prevent overfitting. A Dropout layer (rate 0.4) is then introduced to enhance the robustness of the learned features. The head ends with a final Dense output layer (29 units) utilizing the Softmax activation function to yield a normalized probability distribution across the target classes.

2.2.3 Vision Transformer

The third and final model we implemented is the Vision Transformer (ViT), an architecture that handles images by fundamentally treating them as sequences of data. Our code builds this model using three main custom components: Patches, PatchEncoder, and ViTBlock.

The process starts with the Patches layer, which takes the 2D input image and breaks it down into a grid of non-overlapping patches. These patches are then flattened into 1D vectors, essentially transforming the image into a linear sequence of tokens.

The PatchEncoder layer then prepares these tokens for the Transformer engine through two critical steps: Patch Embedding and Positional Encoding. The first one, which is a simple linear projection, maps the high-dimensional patch data down to a consistent embedding dimension. Then, because the standard Transformer design has no built-in spatial awareness, we generate and add a set of learnable positional embeddings to the patch embeddings. This gives the model the necessary context to understand where each patch sits in the original image, allowing it to model relationships based on proximity.

Then, the core of the ViT is the ViTBlock, which is repeated multiple times (specifically, 12). Each block is built around two key mechanisms: Multi-Head Attention (MHA) and a Feed-Forward Network (FFN). The MHA mechanism allows each patch token to globally attend to all other patches in the sequence, dynamically determining the most relevant image parts. Both sub-layers are stabilized using Layer Normalization and include Residual Connections to ensure smooth training. As we did in other models, we also use Dropout to prevent overfitting.

After the sequence of patches has been processed by the Transformer blocks, a final classification head is used to produce the prediction. This last layer involves applying GlobalAveragePooling1D to reduce the entire sequence of patch embeddings down to a single feature vector. Finally, this vector passes through a Dropout layer for regularization and a Dense output layer with softmax activation, which produces the final probability distribution over the 29 targets.

3 Result Analysis

3.1 Standard Architecture

To evaluate the standard architectures, the same set of hyperparameters was applied to both models, without altering their structural components or layers. This decision was made to ensure that any differences in performance arise from the architectures themselves rather than from changes in design. Three different configurations were used for each model, corresponding to underfitting, overfitting, and a balanced scenario.

The **underfitting** configuration employs a relatively large batch size of 64 and a very small learning rate of 1.2×10^{-5} , limiting the network’s ability to update its weights effectively. Training is performed for 60 epochs, and the learning rate decays quickly with a step of 5 and a factor of 0.58, while the optimizer betas are set high (0.988, 0.9988) to provide smooth momentum. Together, these settings have been

chosen to restrict the model’s capacity to fully capture the patterns in the dataset in order to produce an underfitting behavior.

In contrast, the **overfitting** configuration is designed to encourage the network to closely fit the training data, even at the risk of reduced generalization. A small batch size of 16 and a higher learning rate of 1×10^{-3} allow the model to learn aggressively, while a longer decay step of 30 with a factor of 0.95 gradually slows learning over 110 epochs. The optimizer betas (0.9, 0.999) support faster adaptation to the gradient, further facilitating memorization of the training set.

The **balanced** configuration represents an intermediate setup aimed at achieving good generalization and a smaller gap between train and test performance. With a moderate batch size of 32 and learning rate of 9×10^{-4} , the network can learn efficiently without extreme oscillations or premature convergence. The learning rate decays every 7 epochs by a factor of 0.68, while the optimizer betas (0.91, 0.991) provide a compromise between smooth momentum and responsiveness.

Table 1: Hyperparameters for Standard Architectures

Parameter	Balanced*	Underfitting	Overfitting
Batch size	32	64	16
Epochs	100	60	110
Learning rate	9×10^{-4}	1.2×10^{-5}	1×10^{-3}
LR step	7	5	30
LR decay	0.68	0.58	0.95
Optimizer betas	(0.91, 0.991)	(0.988, 0.9988)	(0.9, 0.999)

* For AlexNet, the balanced configuration used 60 epochs instead of 100.

The **underfitting** configuration reveals clear differences in how both architectures respond to severely constrained learning conditions. AlexNet demonstrates complete training failure, with both training and validation accuracies failing to exceed 10% throughout the entire training process. In contrast, the StandardArchitectureModel achieves approximately 50% accuracy on both training and validation sets, with validation accuracy slightly exceeding training accuracy, a clear sign of severe underfitting. For this problem, a 50% accuracy significantly exceeds random guessing but still suggesting that the model learned basic patterns but plateaued early. The stability of metrics around 50% from epoch 20 onwards confirms that the model reached its learning ceiling under these restrictive hyperparameters..

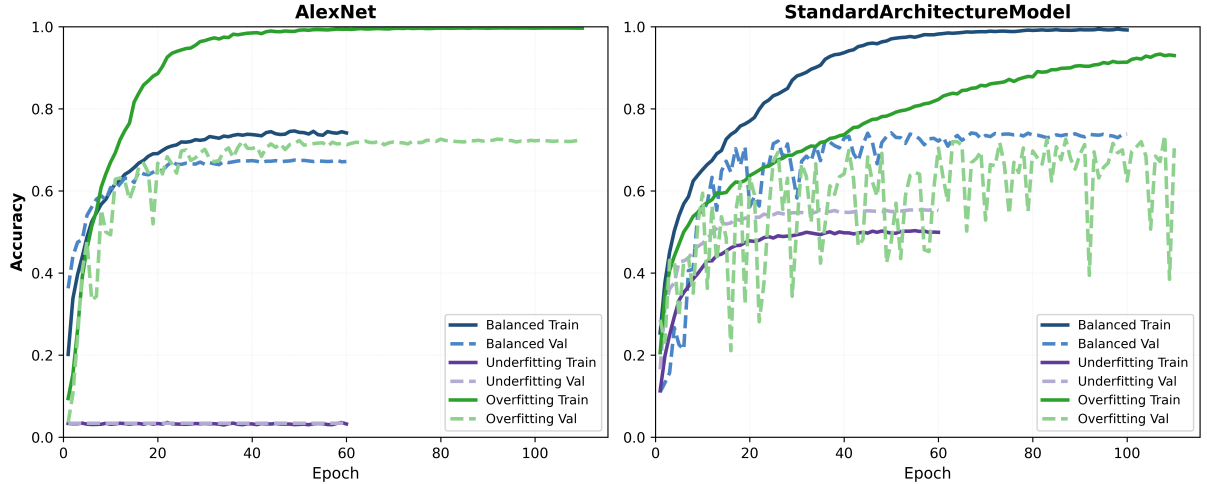


Figure 1: Comparison plot for all three standard architectures on the three configurations: overfitting, underfitting and balanced

The **overfitting** configuration produces dramatically different training dynamics, although neither architecture achieves the classical overfitting behavior. AlexNet shows rapid initial learning, with training accuracy quickly surpassing 95%, but extreme volatility in both metrics with validation accuracy oscillating between 40% and 70% indicates training instability rather than true overfitting. The Standard Architecture Model exhibits similar behavior, reaching approximately 92% training accuracy while validation accuracy stabilises below 70% with some fluctuations. This fluctuation pattern, combined with

the lack of metric convergence, suggests gradient instability caused by an excessively high learning rate rather than memorization of the training data. This behaviour is not exactly true overfitting as it would be characterized by high and stable training accuracy alongside lower but stable validation accuracy, which is not exactly the case for AlexNet.

The **balanced** configuration demonstrates successful training for both architectures, with distinct generalization characteristics. AlexNet exhibits textbook balanced behavior, reaching 74% training accuracy and 66.5% validation accuracy by epoch 40, maintaining a stable gap of approximately 7.5 percentage points through epoch 60. The Standard Architecture Model presents a more nuanced case: training accuracy approaches 100% while validation accuracy plateaus at 73.9%, creating a larger gap of around 26 percentage points. Despite this gap, the configuration remains balanced for several reasons. First, validation accuracy remains high and stable from epoch 60 onwards, with no degradation, indicating that the model is not progressively memorizing the training data. Second, the stability observed across the final 40 epochs demonstrates convergence to a solution that generalizes well, even with very high training performance.

If we take a deeper look at the results of the Standard Architecture Model, concretely those on Figure 2, the **balanced** configuration can be considered balanced because the training loss decreases smoothly from approximately 7.5 to near 0 by epoch 100, while the validation loss drops from around 5.5 and stabilizes at 0.88 by epoch 30, showing minimal fluctuations. Validation accuracy plateaus at 73.9%, with steady improvement up to epoch 60 and no subsequent degradation, indicating convergence to a generalizable solution rather than memorization. The learning rate decays from 9×10^{-4} to about 6×10^{-6} in stepwise reductions every 20 epochs, allowing rapid early convergence and fine-tuning later. The overall stability of loss and validation accuracy across the final 40 epochs confirms effective learning and strong generalization without overfitting.

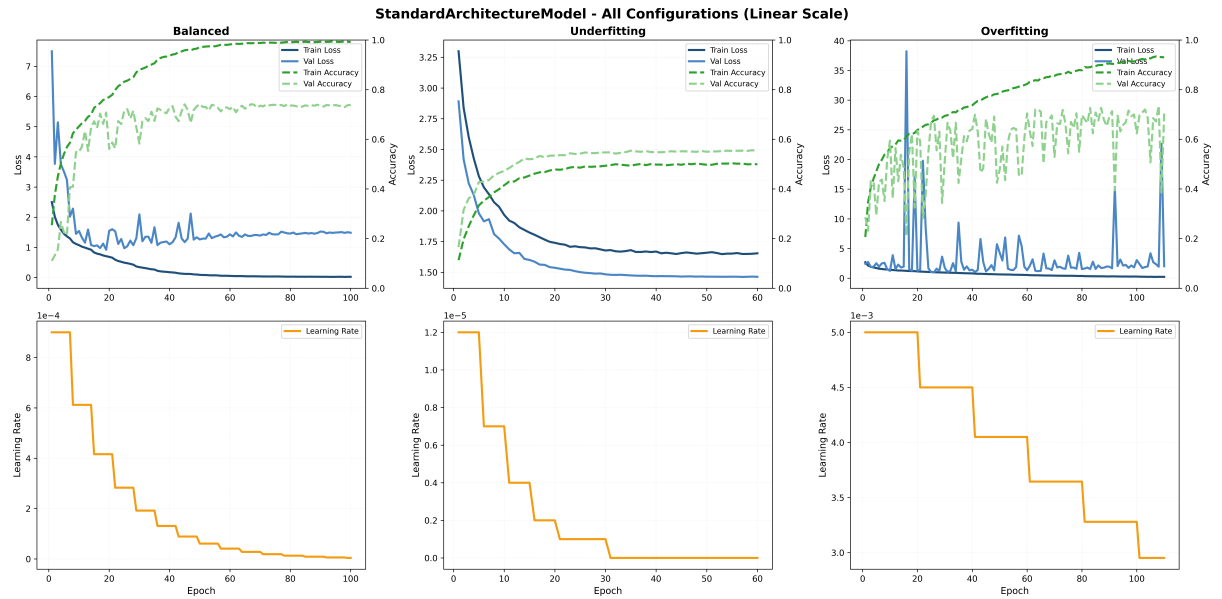


Figure 2: Detailed accuracy and loss plot for the standard architecture

The **underfitting** configuration shows the effect of an excessively low learning rate on the model’s training capacity. Training and validation losses decrease slowly, with training loss falling from 3.25 to around 1.65 and validation loss from 2.9 to 1.48 by epoch 60, yet neither reaches the lower values seen in the balanced configuration. Validation accuracy stabilizes around 50%, matching the training accuracy and occasionally slightly exceeding it, a clear sign of severe underfitting where the model cannot effectively capture the data patterns. This behavior is caused by the extremely low initial learning rate of 1.2×10^{-5} about 75 times smaller than in the balanced configuration which decays rapidly to near zero by epoch 30, effectively halting weight updates. For a 29 class problem, where random guessing would yield roughly 3.4% accuracy, the 50% achieved indicates the model learned only basic patterns but is unable to progress further due to constrained optimization dynamics.

The **overfitting** configuration shows some training instability, as previously mentioned. Training loss and validation accuracy show some fluctuation. Training accuracy rises quickly to approximately 92% but fluctuates between 60% and 92%, and validation accuracy varies widely between 20% and 70% with no

clear convergence. This instability is caused by the learning rate schedule, which starts at 5×10^{-3} about 5.5 times higher than in the balanced configuration and decays slowly to 3×10^{-3} by epoch 100. The excessively large updates cause unstable training, with both training and validation metrics fluctuating. While not classical overfitting, the model still attempts to fit the training data too aggressively, leading to intermittent spikes in accuracy and loss. This shows that overfitting can occur with high instability when the learning rate is excessively large, producing poor generalization too.

3.2 Non-standard Architecture

Unlike the approach used for the standard architectures, the unique nature of the different non-standard architectures required a flexible tuning strategy. Each architecture possessed distinct structural requirements, meaning that a universal hyperparameter set was suboptimal. Finally, we adhered to the requirement of the practical exercise: defining three distinct configurations for every model, corresponding to underfitting, overfitting, and a balanced scenario.

The tables below summarize the hyperparameter configurations used for the executions of the InceptionV3 and ViT Model architectures.

Table 2: Hyperparameters for InceptionV3 Model

Parameter	Balanced*	Underfitting	Overfitting
Batch size	32	16	256
Epochs	100	25	50
Learning rate	5×10^{-4}	1×10^{-6}	5×10^{-2}
LR step	15	50	10
LR decay	0.85	1	1
Momentum	0.9	0	0.9

Regarding the InceptionV3 configuration from Table 2, the **underfitting** setup uses hyperparameters that severely restrict learning. The extremely small learning rate (1×10^{-6}) prevents meaningful weight updates, and zero momentum offers no acceleration. With only 25 epochs and an LR Step set to 50, which means that the model never triggers decay, the model effectively stops improving early.

For the **overfitting** configuration, the hyperparameters are tuned to push the model toward memorizing the training set, increasing the overfitting risk. The high learning rate enables large weight updates, and the large batch size produces rapid gradients. With 50 epochs and no LR decay, the model continues learning aggressively, while the 0.9 momentum sustains this fast pace.

Finally, the **balanced** configuration targets a stable middle ground. It uses a moderate batch size and a reasonable learning rate (5×10^{-4}), enabling efficient but stable training. With 100 epochs and a decay of 0.85 every 15 epochs, the learning rate decreases gradually, avoiding premature convergence. The momentum value set to 0.9 supports smooth learning and good generalization.

Table 3: Hyperparameters for ViT Model

Parameter	Balanced*	Underfitting	Overfitting
Batch size	64	256	256
Epochs	75	120	50
Learning rate	5×10^{-4}	1×10^{-3}	1×10^{-3}
LR step	20	10	10
LR decay	0.7	0.1	1

For the ViT **underfitting** setup detailed in Table 3, the very aggressive regularization - specifically the sharp LR Decay (0.1 every 10 steps) - forces the learning rate to drop too quickly, stopping meaningful updates early. Even with a large batch size and a reasonable initial learning rate, 1×10^{-3} , the rapid decay prevents the model from converging, leading to underfitting.

To induce **overfitting**, training uses 50 epochs with no LR decay, keeping the 1×10^{-3} learning rate constant. The large batch size provides stable gradients, enabling the model to consistently fit the training data. Together, the steady learning rate and reliable updates encourage weak generalization.

Lastly, the **balanced** configuration uses a moderate batch size, a cautious learning rate (5×10^{-4}), and 75 training epochs. A decay factor of 0.7 every 20 steps steadily lowers the learning rate, allowing the ViT to learn reliably without drifting into non-convergence or overfitting.

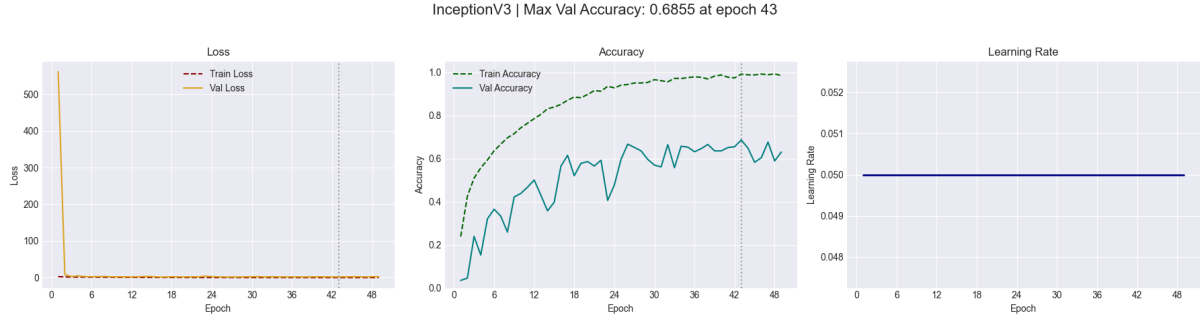


Figure 3: Detailed accuracy, loss and learning rate plot for the Inception Model; overfitting

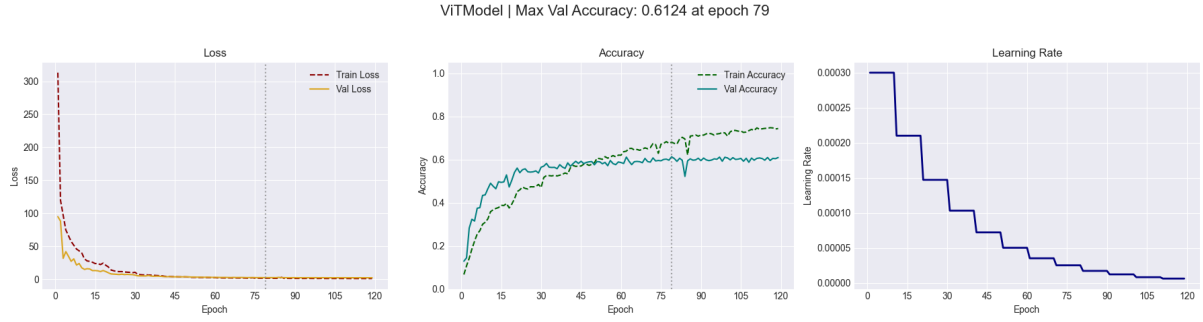


Figure 4: Detailed accuracy, loss and learning rate plot for the ViT Model; overfitting

Figures 3 and 4 show that both architectures achieve **overfitting**, but in different ways. InceptionV3 displays extreme overfitting, with a rapidly widening gap between training and validation accuracy and highly unstable validation loss, reflecting destructive volatility. In contrast, the ViT model demonstrates a more controlled pattern: its moderate, constant learning rate allows validation loss to drop to a minimum before rising gradually, while training accuracy continues to improve, nearly reaching 0.8% as validation stabilizes around 0.6%. This behavior indicates that ViT, though stable, ultimately fails to generalize, confirming its limitations in capturing patterns beyond the training data.

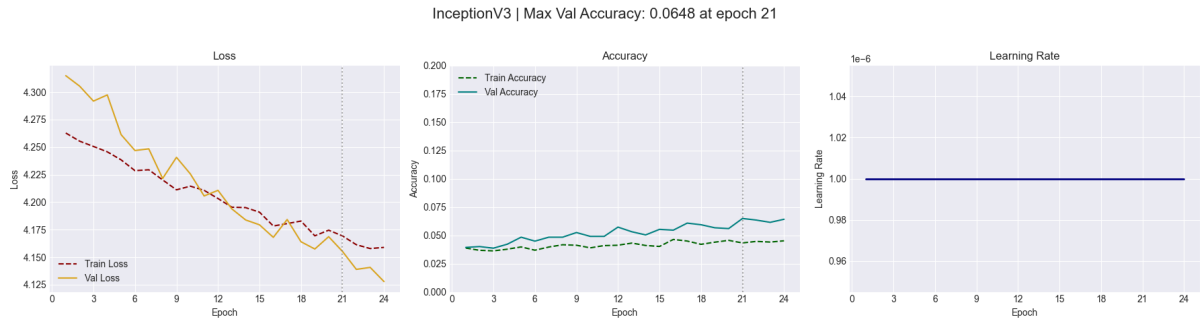


Figure 5: Detailed accuracy, loss and learning rate plot for the Inception Model; underfitting

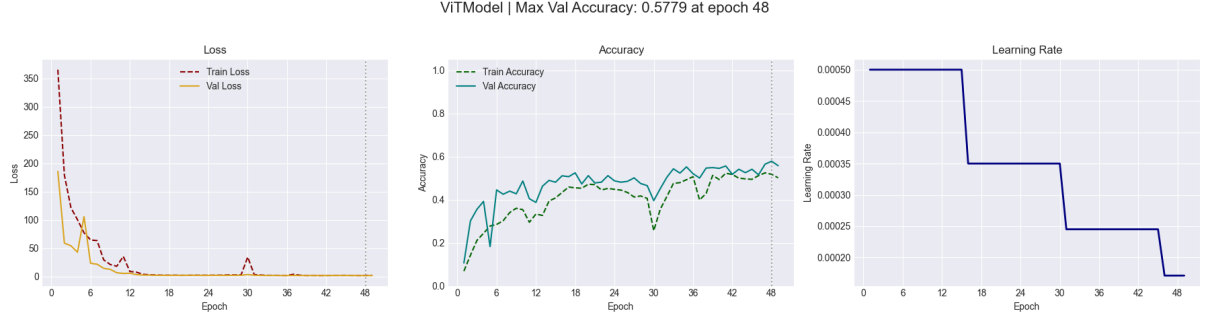


Figure 6: Detailed accuracy, loss and learning rate plot for the ViT Model; underfitting

Again, the two architectures exhibit distinct **underfitting** types, showing how different hyperparameter shortcomings affect performance. InceptionV3 shows severe underfitting in 5, with Training and validation accuracy remaining below 7% and flat, while loss curves stay high, reflecting optimization paralysis. This arises from the extremely low learning rate (1×10^{-6}), which effectively froze weight updates. The ViT model shows a gentler **underfitting** 6, with accuracy initially rising to 50% before stopping. Loss curves drop then plateau near zero, indicating the cease of effective learning due to aggressive learning rate decay (0.1 every 10 steps), which limited updates and probably prevented full convergence.

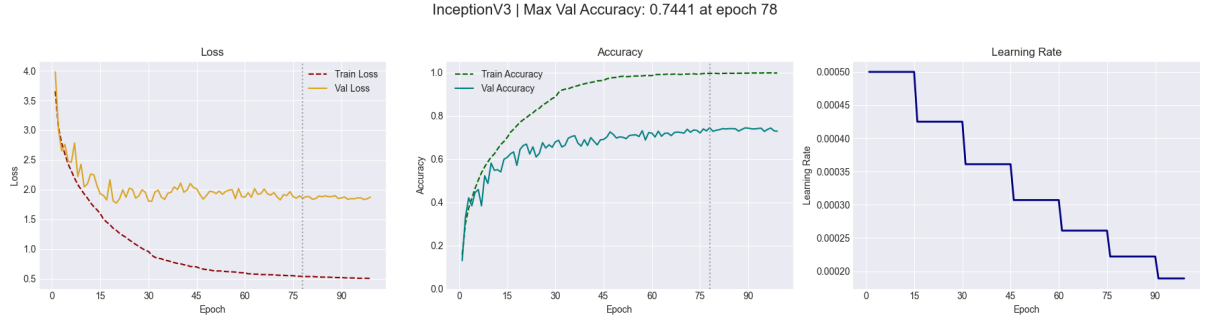


Figure 7: Detailed accuracy, loss and learning rate plot for the Inception Model; balanced

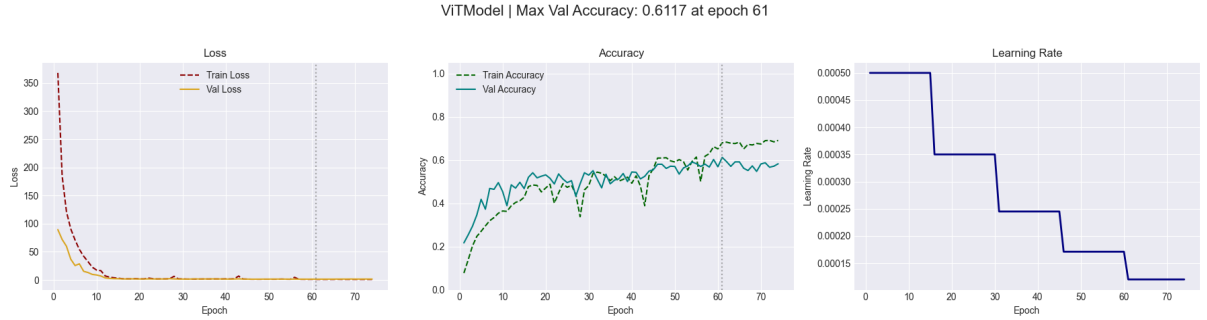


Figure 8: Detailed accuracy, loss and learning rate plot for the ViT Model; balanced

The two **balanced** configurations generated strong, generalizable models, but with key performance differences. Figure 7 demonstrated how the InceptionV3 model managed to achieved a significantly high maximum validation accuracy of 0.75% and exhibited a smooth, more predictable convergence. Its performance suggests that the architecture, combined with the gradual learning rate decay, was highly effective and stable for the task. The loss curves clearly showed the point where generalization peaked and training loss began to diverge. In contrast, Figure 8 with the metrics for the ViT Model shows how the model achieved a slightly lower validation accuracy (almost reaching 0.6%) and its training was characterized by higher volatility and noise in the accuracy curves. However, the gap between its training

and validation accuracy was visually smaller than InceptionV3's, suggesting that while it did not reach as high accuracy results, its generalization was tighter. This volatility is likely related to the sharper nature of its learning rate decay, which caused more abrupt changes in the optimization trajectory. Overall, InceptionV3 proved to be the more performant and stable architecture for this specific task under the balanced scenario.

4 Conclusions

The practical work successfully demonstrated the impact of architectural choices and hyperparameter tuning on model generalization using the MAMe dataset. The core objective of generating distinct underfitting, overfitting, and balanced scenarios was met for both the Standard (AlexNet and Custom Standard) and Non-Standard (InceptionV3 and ViT) CNN architectures. Although ResNet18 was also implemented, its results mainly reflected extreme overfitting - either reduced or exaggerated depending on the configuration - without offering additional insights beyond those already observed in the other models. For this reason, its results were not included in the report, though the full implementation remains available in the project code.

Comparing the four architectures, the custom Standard Architecture Model and InceptionV3 were the top performers, showing similar results. Both exhibited early fluctuations in validation accuracy that stabilized around epochs 45-50. The Custom Standard Architecture Model showed slightly more volatile loss, with occasional peaks, but achieved marginally higher accuracy than InceptionV3, though neither reached 80%.

Finally, due to the focus of the exercise being on architectural comparisons and training dynamics, detailed classification analysis was limited. Specifically, full performance metrics such as confusion matrices and per-class classification metrics were not analyzed. Furthermore, the training process for these complex models was time-intensive, requiring several hours per execution. Consequently, execution time and training efficiency were not measured. Finally, external tools such as ChatGPT were used to assist in generating and formatting the visual plots included in this report, as well as in refining certain parts of the text for clarity and a more formal style. It is important to note, however, that these tools were not used to produce original ideas or reasoning.